

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E
TECNOLOGIA DE SÃO PAULO
CAMPUS VOTUPORANGA

JOÃO PEDRO DOS SANTOS SOUZA

**APLICAÇÃO WEB PARA GERAÇÃO DE DOCUMENTOS EM MASSA COM
CAMPOS DINÂMICOS E INTEGRAÇÃO A APIS**

VOTUPORANGA

2026

João Pedro dos Santos Souza

**APLICAÇÃO WEB PARA GERAÇÃO DE DOCUMENTOS EM MASSA COM
CAMPOS DINÂMICOS E INTEGRAÇÃO A APIS**

Trabalho de Conclusão de Curso
apresentado como exigência parcial para
obtenção do diploma do Curso em
Bacharelado em Sistemas de Informação
do Instituto Federal de Educação, Ciência
e Tecnologia, Câmpus Votuporanga.

Professor Orientador: Eduardo de Pieri
Prando.

DEDICATÓRIA

*Dedico este trabalho a todos os colegas
de classe que estiveram presentes
nessa trajetória de alegrias e superação.*

AGRADECIMENTOS

A todos os professores e servidores do IFSP Campus Votuporanga que contribuíram direta e indiretamente para a conclusão desse trabalho.

À minha família, que deu todo o apoio necessário para que eu chegasse até aqui.

Ao meu orientador, que me auxiliou a solucionar as dificuldades encontradas no caminho.

EPÍGRAFE

*"O que prevemos raramente ocorre; o
que menos esperamos geralmente
acontece".*

Benjamin Disraeli

RESUMO

O trabalho em questão aborda o problema da ineficiência nos processos de negócios causado pela criação de manual, propondo como objetivo o desenvolvimento de uma aplicação web para automação e geração dinâmica de conteúdo. A metodologia consiste na construção de um sistema baseado em templates inteligentes, de fácil uso, que utilizam uma linguagem de marcação para inserção de dados variáveis e um motor de cálculo para operações dinâmicas. A arquitetura tecnológica é fundamentada em Node.js para o backend e utiliza o MongoDB como banco de dados, garantindo uma solução flexível e escalável. Como resultado, espera-se apresentar um protótipo funcional que valida a viabilidade da solução, demonstrando seu potencial para reduzir erros, otimizar o tempo e padronizar a documentação em diversos setores do mercado. O projeto contribui com uma solução prática para um problema de negócio relevante e estabelece uma base para futuras evoluções, como a integração com uma IA personalizada para dar assistência ao usuário.

Palavras-chave: automação de documentos; geração dinâmica de documentos; Node.js; sistemas de informação; transformação digital.

ABSTRACT

This work addresses the problem of inefficiency in business processes caused by manual creation, proposing as its objective the development of a web application for the automation and dynamic generation of content. The methodology consists of building a system based on user-friendly, intelligent templates that utilize a markup language for variable data insertion and a calculation engine for dynamic operations. The technological architecture is founded on Node.js for the backend and uses MongoDB as the database, ensuring a flexible and scalable solution. As a result, the expected outcome is a functional prototype that validates the solution's feasibility, demonstrating its potential to reduce errors, optimize time, and standardize documentation across various market sectors. The project contributes a practical solution to a relevant business problem and establishes a foundation for future evolutions, such as the integration with a custom AI to provide user assistance.

Keywords: document automation; dynamic document generation; Node.js; information systems; digital transformation.

LISTA DE ILUSTRAÇÕES

Figura 1 – Diagrama de Caso de Uso.....	41
Figura 2 – Diagrama de Classes.....	42

LISTA DE TABELAS

Tabela 1 – Comparativo entre Gestão Documental Manual e Automatizada.....	30
Tabela 2 – Análise Comparativa: Modelo Relacional (SQL) vs. Modelo de Documentos (MongoDB) para Armazenamento de Templates.....	33

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
IFSP	Instituto Federal de Educação, Ciência e Tecnologia de São Paulo

LISTA DE SÍMBOLOS

d_{ab}	Distância euclidiana
$O(n)$	Ordem de um algoritmo

SUMÁRIO

1. INTRODUÇÃO	25
2. Objetivos	27
2.1. Objetivo Geral	27
2.2. Objetivos específicos	27
3. Revisão de literatura	28
3.1. FUNDAMENTOS PARA A AUTOMAÇÃO INTELIGENTE DE DOCUMENTOS	28
3.2. O Cenário Estratégico: Transformação Digital e a Otimização de Processos de Negócio	28
3.3. A Gênese do Problema: Ineficiências e Riscos da Gestão Documental Manual	29
3.4. O Núcleo da Solução: O Paradigma dos Motores de Template para Geração Dinâmica	31
3.5. Arquitetura de Suporte para uma Aplicação Web Moderna e Escalável	33
3.5.1. Backend reativo com Node.js: O modelo orientado a eventos	33
3.5.2. Persistência de dados flexível com MongoDB: O poder do NoSQL ...	34
3.5.3. Segurança e autenticação stateless via JWT	35
3.6. Conformidade, Integrações e Horizontes Futuros	36
3.6.1. Implicações da Lei Geral de Proteção de Dados (LGPD)	36
3.6.2. Extensibilidade e o ecossistema de APIs	37
3.6.3. A próxima fronteira – Processamento inteligente de documentos (IDP)	
38	
4. Material e métodos	39
4.1. NATUREZA E CLASSIFICAÇÃO DA PESQUISA	39
4.1.1. Quanto à natureza	39
4.1.2. Quanto à abordagem do problema	39
4.1.3. Quanto aos objetivos	39
4.1.4. Quanto aos procedimentos técnicos	40

4.2.	ARQUITETURA DO SISTEMA E FERRAMENTAS UTILIZADAS	40
4.2.1.	Backend e Lógica de Negócios (Node.js).....	40
4.2.2.	Armazenamento e Persistência de Dados (MongoDB)	40
4.2.3.	Motor de Templates (Handlebars.js).....	41
4.2.4.	Segurança e Autenticação (JWT).....	41
4.3.	MODELAGEM DO SISTEMA	41
4.3.1.	Diagrama de Casos de Uso	42
4.3.2.	Diagrama de Classes	43
4.4.	ETAPAS DE DESENVOLVIMENTO	44
4.4.1.	Levantamento e Análise de Requisitos	44
4.4.2.	Implementação e Codificação (Construção do MVP)	44
4.4.3.	Testes e Validação do Sistema	45
5.	resultados e discussão	46
6.	CONSIDERAÇÕES FINAIS	48
7.	Sugestão trabalhos futuros	49
	REFERÊNCIAS	50

1. INTRODUÇÃO

A transformação digital tornou-se um pilar fundamental para a competitividade e eficiência das organizações no cenário contemporâneo. Dentro deste contexto, a Automação de Processos de Negócio (BPA) destaca-se como uma área estratégica, e um de seus componentes mais impactantes é a automação de documentos, que visa substituir a criação manual e repetitiva de documentos por fluxos de trabalho digitais e inteligentes. A criação de contratos, propostas comerciais, laudos técnicos e relatórios, quando realizada manualmente, representa um gargalo operacional significativo, sendo um processo lento, propenso a erros humanos e que dificulta a padronização. Este trabalho se localiza no campo teórico da otimização de processos por meio de Sistemas de Informação, abordando diretamente este desafio. O objetivo geral desta pesquisa é projetar e desenvolver uma plataforma *web* para a automação e geração dinâmica de documentos. Como objetivos específicos, busca-se: construir um motor de *templates* que utilize uma linguagem de marcação simples para a inserção de dados variáveis que todo usuário entende com facilidade; implementar uma linguagem de cálculo personalizada para a execução de operações matemáticas e lógicas nos documentos; e desenvolver uma interface intuitiva que permita a usuários sem conhecimento técnico gerenciar seus próprios modelos de automação.

Para atingir os objetivos propostos, a metodologia adotada será a de desenvolvimento de um protótipo funcional (MVP) de uma aplicação *web*, com potencial para operar como um serviço (SaaS). A arquitetura tecnológica selecionada para o projeto inclui o uso de Node.js para o desenvolvimento do *backend*, garantindo uma API ágil e escalável; o banco de dados NoSQL MongoDB, escolhido por sua flexibilidade para armazenar as estruturas dinâmicas dos *templates*; e a biblioteca Handlebars.js para a renderização do *frontend*, que se alinha perfeitamente com a lógica de marcação do sistema. A segurança e a autenticação dos usuários serão gerenciadas por meio de um padrão de mercado, o JWT.

A justificativa para a realização deste trabalho reside na sua alta relevância tanto acadêmica quanto de mercado. Dados de consultorias como a Gartner (2023) e a McKinsey & Company (2022) apontam que o mercado global de automação de documentos está em franca expansão, impulsionado pela necessidade de reduzir custos operacionais e mitigar os riscos associados a erros humanos, que podem gerar prejuízos financeiros e legais significativos. No contexto brasileiro, a crescente

digitalização das empresas e a vigência de regulamentações como a Lei Geral de Proteção de Dados (LGPD) aumentam a demanda por processos documentais que sejam padronizados, seguros e auditáveis. Dessa maneira, a plataforma proposta não apenas representa a aplicação prática de conceitos de engenharia de software e sistemas de informação, mas também oferece uma solução concreta para um problema de negócio real, com o potencial de liberar profissionais de tarefas de baixo valor agregado para que possam focar em atividades mais estratégicas, aumentando a produtividade e a competitividade em diversos setores da economia.

2. OBJETIVOS

2.1. OBJETIVO GERAL

O objetivo geral desta pesquisa é projetar e desenvolver uma plataforma web funcional para a automação e geração dinâmica de documentos, oferecendo uma solução prática para otimização de processos em ambientes corporativos.

2.2. OBJETIVOS ESPECÍFICOS

Para que o objetivo geral seja alcançado, os seguintes objetivos específicos foram traçados:

- Construir um motor de templates que utilize uma linguagem de marcação simples para a inserção de dados variáveis;
- Implementar uma linguagem de cálculo personalizada para a execução de operações matemáticas e lógicas diretamente nos documentos;
- Desenvolver uma interface de usuário intuitiva que permita a pessoas sem conhecimento técnico avançado criar e gerenciar seus próprios modelos de automação.

3. REVISÃO DE LITERATURA

3.1. FUNDAMENTOS PARA A AUTOMAÇÃO INTELIGENTE DE DOCUMENTOS

Este capítulo estabelece a fundamentação teórica que sustenta o desenvolvimento da plataforma de automação e geração dinâmica de documentos. A estrutura parte do contexto macroestratégico da Transformação Digital, afunilando para o problema específico da gestão documental manual, detalhando a solução tecnológica central e sua arquitetura de suporte, e, por fim, posicionando o projeto em seu ecossistema regulatório e de inovação futura.

3.2. O CENÁRIO ESTRATÉGICO: TRANSFORMAÇÃO DIGITAL E A OTIMIZAÇÃO DE PROCESSOS DE NEGÓCIO

A plataforma proposta neste trabalho não é um artefato tecnológico isolado, mas uma resposta direta às forças que moldam o ambiente de negócios contemporâneo. Para compreender sua relevância, é imperativo contextualizá-la dentro de duas macrotendências interligadas: a Transformação Digital (TD) e a Automação de Processos de Negócio (BPA).

A Transformação Digital transcende a mera adoção de novas tecnologias; ela representa uma mudança fundamental nos modelos de negócio, nas operações e na cultura organizacional, impulsionada pela combinação estratégica de tecnologias digitais. Trata-se de uma reconfiguração na forma como as organizações pensam e agem para se manterem competitivas e disruptivas em um mercado em constante evolução. Este projeto, ao propor uma ferramenta que democratiza a automação para usuários sem conhecimento técnico aprofundado, atua como um catalisador para essa mudança cultural, alinhando-se ao cerne da TD.

Dentro deste amplo espectro, a Automação de Processos de Negócio (do inglês, *Business Process Automation* - BPA) emerge como um pilar tático fundamental. A consultoria Gartner define BPA como "a automação de processos e funções de negócios complexos que vão além da manipulação de dados convencional e das atividades de manutenção de registros, geralmente através do uso de tecnologias avançadas". Em essência, a BPA utiliza software para automatizar

transações em múltiplas etapas que são repetitivas, conectando diversos sistemas de TI para otimizar fluxos de trabalho e aumentar a eficiência operacional.

A implementação da BPA não é apenas uma medida de otimização, mas uma estratégia competitiva. Ao automatizar processos, as empresas conseguem entregar resultados mais rapidamente e com maior qualidade, melhorando a experiência do cliente e a capacidade de resposta às mudanças do mercado. A relevância desta abordagem é refletida no crescimento do mercado global de BPA, que, impulsionado pela busca incessante por eficiência e redução de custos, tem projeções de crescimento que alcançam dezenas de bilhões de dólares.

É possível, portanto, estabelecer uma clara hierarquia conceitual que posiciona este trabalho em um contexto de alta relevância estratégica. A Transformação Digital funciona como a estratégia global que orienta a modernização de uma organização. A Automação de Processos de Negócio é uma das principais táticas empregadas para executar essa estratégia, focando na otimização de fluxos de trabalho específicos. A plataforma de automação de documentos aqui proposta, por sua vez, é uma ferramenta especializada e crucial para a implementação efetiva da BPA. Ela não é apenas "um sistema que gera documentos", mas um habilitador tático que viabiliza a execução da estratégia maior de transformação digital, resolvendo um problema operacional com profundas implicações estratégicas.

3.3. A GÊNESE DO PROBLEMA: INEFICIÊNCIAS E RISCOS DA GESTÃO DOCUMENTAL MANUAL

A necessidade de automação de documentos origina-se das deficiências intrínsecas aos processos manuais, que representam um gargalo significativo para a produtividade, segurança e conformidade das organizações modernas. A dependência de métodos tradicionais para criar, gerenciar e armazenar documentos gera um custo oculto que vai muito além dos gastos com papel e impressão, configurando o que pode ser descrito como um "gap silencioso" na performance organizacional.

A primeira e mais evidente desvantagem é a ineficiência e perda de produtividade. Processos manuais são inerentemente lentos, repetitivos e consomem um tempo valioso de profissionais que poderiam estar alocados em atividades de maior valor estratégico. Pesquisas apontam que gestores podem perder o equivalente

a um mês de trabalho por ano simplesmente procurando informações extraviadas, e colaboradores chegam a gastar mais de 50% de seu tempo em busca de dados. Um estudo específico sobre sistemas de contrato revelou que processos manuais são responsáveis por 61% do retrabalho e 43% dos atrasos em projetos, números que quantificam o impacto direto na agilidade operacional.

Em segundo lugar, a intervenção humana contínua em tarefas repetitivas é um terreno fértil para erros. Fatores como cansaço, falta de atenção ou simples negligência podem levar a equívocos em contratos, relatórios e propostas. Embora dados de outros setores, como o industrial, indiquem que o erro humano pode ser a causa de 80 a 90% dos acidentes, o princípio se aplica universalmente: a falibilidade humana é um risco inerente a qualquer processo manual. No contexto documental, esses erros podem resultar em prejuízos financeiros diretos, litígios, retrabalho e danos à reputação da empresa.

Adicionalmente, a gestão manual acarreta graves riscos de segurança e conformidade. Documentos físicos são vulneráveis a extravio, danos e acesso não autorizado, enquanto arquivos digitais espalhados em e-mails e planilhas carecem de controle centralizado. Em um cenário regulatório cada vez mais rigoroso, essa vulnerabilidade representa um risco legal e financeiro iminente. No Brasil, a Lei Geral de Proteção de Dados (LGPD) impõe sanções severas para o tratamento inadequado de dados pessoais, com multas que podem atingir até R\$ 50 milhões. A automação, com seus recursos de controle de acesso e rastreabilidade, torna-se, portanto, um componente essencial para a estratégia de conformidade de qualquer organização.

Por fim, a falta de padronização é uma consequência inevitável da criação manual. Documentos elaborados por diferentes colaboradores em momentos distintos tendem a apresentar inconsistências de formato, linguagem e estrutura, o que não apenas dificulta a gestão, mas também enfraquece a identidade visual e a comunicação da marca.

O problema que este projeto se propõe a resolver, portanto, não é um mero inconveniente, mas uma vulnerabilidade estratégica. A plataforma ataca diretamente esse "gap silencioso" ao converter tempo desperdiçado em produtividade estratégica e ao mitigar riscos latentes por meio da padronização, segurança e controle.

Tabela 1 – Comparativo entre Gestão Documental Manual e Automatizada.

Métrica	Gestão Documental Manual	Gestão Automatizada com a Plataforma Proposta
Produtividade e Tempo	Processo lento, consome horas de trabalho qualificado, gera atrasos e retrabalho.	Geração instantânea de documentos, liberando a equipe para tarefas estratégicas e de maior valor.
Precisão e Taxa de Erros	Alta suscetibilidade a erros humanos por digitação, omissão ou inconsistência.	Redução drástica de erros ao preencher dados a partir de uma fonte única e validada, eliminando a entrada manual.
Custos Operacionais	Custos diretos (impressão, armazenamento) e indiretos (tempo perdido, retrabalho, multas).	Redução de custos com insumos e, principalmente, otimização do tempo da equipe, aumentando a eficiência geral.
Segurança e Conformidade (LGPD)	Risco elevado de vazamentos, acesso não autorizado e dificuldade em auditar o manuseio de dados.	Controle de acesso granular, trilhas de auditoria e padronização de cláusulas, facilitando a conformidade com a LGPD.
Padronização e Consistência	Documentos inconsistentes em formato, linguagem e identidade visual, dependendo do autor.	Garantia de padronização total, fortalecendo a identidade da marca e a clareza da comunicação.
Escalabilidade	Dificuldade em lidar com o aumento do volume de documentos sem aumentar proporcionalmente a equipe.	Alta escalabilidade, permitindo a geração de um grande volume de documentos sem impacto na performance ou custo marginal.

Fonte: Elaborado pelo autor (2026).

3.4. O NÚCLEO DA SOLUÇÃO: O PARADIGMA DOS MOTORES DE TEMPLATE PARA GERAÇÃO DINÂMICA

A transição do problema para a solução tecnológica passa pela compreensão do conceito central que viabiliza a geração dinâmica de documentos: o motor de template (*template engine*). Esta ferramenta de software é o coração da plataforma proposta, permitindo a fusão eficiente entre modelos pré-definidos e dados variáveis.

Um motor de template é, por definição, um software projetado para combinar *templates* (modelos com uma estrutura fixa e marcadores de posição) com um *modelo de dados* (as informações variáveis) para produzir documentos finais. A principal

função e benefício de um motor de template é promover a separação de responsabilidades entre a lógica da aplicação e a camada de apresentação. Essa separação é um princípio fundamental da engenharia de software, consagrado em padrões de arquitetura como o *Model-View-Controller* (MVC), onde a *View* (o template) é responsável apenas por exibir os dados que recebe, sem se preocupar com a forma como esses dados foram obtidos ou processados.

Para a implementação deste projeto, foi selecionada a biblioteca Handlebars.js, um motor de template popular, conhecido por sua simplicidade semântica e por sua filosofia "logic-less" (sem lógica complexa embutida). Sendo amplamente compatível com a sintaxe do Mustache, o Handlebars utiliza expressões simples, delimitadas por chaves duplas (ex: `{{nome_cliente}}`), para indicar onde os dados dinâmicos devem ser inseridos no documento final.

O funcionamento do Handlebars.js pode ser resumido em um processo de três etapas principais:

1. **Definição do Template:** O usuário cria um documento modelo (em HTML ou outro formato de texto) contendo marcadores de posição, como `<p>Prezado(a) {{nome}}, portador(a) do CPF {{cpf}}...</p>`.
2. **Compilação:** A biblioteca, através da função `Handlebars.compile()`, analisa este template e o converte em uma função JavaScript otimizada.
3. **Execução (Renderização):** Esta função compilada é então executada, recebendo como argumento um objeto de dados (o "contexto"), como `{ nome: 'Maria Silva', cpf: '123.456.789-00' }`. O retorno é a string final do documento, com todos os marcadores de posição devidamente substituídos pelos valores correspondentes.

A característica "logic-less" do Handlebars.js é de particular importância para este projeto. Essa filosofia de design não deve ser vista como uma limitação, mas como uma escolha deliberada que restringe a complexidade da lógica que pode ser embutida nos templates. Ao focar em substituição de variáveis e estruturas de controle básicas, como laços (`{{#each}}`) e condicionais (`{{#if}}`), a sintaxe permanece limpa, declarativa e, crucialmente, acessível para usuários que não são programadores.

Esta escolha tecnológica está, portanto, intrinsecamente ligada à proposta de valor do sistema. Um dos objetivos específicos do trabalho é "desenvolver uma interface de usuário intuitiva que permita a pessoas sem conhecimento técnico avançado criar e gerenciar seus próprios modelos de automação". A simplicidade

imposta pelo Handlebars.js é o que torna este objetivo viável. Ao contrário de motores de template que permitem a execução de código arbitrário, o Handlebars garante que o usuário final — seja um advogado, um gestor de RH ou um administrador — possa criar e manter seus próprios templates de forma autônoma, sem depender de uma equipe de TI. A tecnologia foi selecionada não por ser a mais "poderosa" em termos de funcionalidade de programação, mas por ser a mais "adequada" e capacitadora para o público-alvo, demonstrando uma abordagem madura e centrada no usuário para o design do sistema.

3.5. ARQUITETURA DE SUPORTE PARA UMA APLICAÇÃO WEB MODERNA E ESCALÁVEL

Para que o motor de template opere de forma eficiente, ele deve ser sustentado por uma arquitetura de software robusta, ágil e escalável. As escolhas tecnológicas para o backend, banco de dados e sistema de autenticação foram feitas para garantir que a plataforma não apenas atenda aos requisitos funcionais, mas também possua os atributos não funcionais essenciais para uma aplicação web moderna.

3.5.1. Backend reativo com Node.js: O modelo orientado a eventos

A base do backend da aplicação é o Node.js, um ambiente de execução JavaScript que se destaca por sua arquitetura orientada a eventos e seu modelo de Entrada/Saída (I/O) não bloqueante. Diferente de modelos tradicionais que alocam uma thread para cada requisição — e que bloqueiam essa thread enquanto aguardam operações lentas como consultas a banco de dados ou leituras de arquivo —, o Node.js opera com um *event loop* de thread única.

Neste modelo, quando uma operação de I/O é iniciada, ao invés de esperar por sua conclusão, o Node.js delega a tarefa ao kernel do sistema operacional (que utiliza múltiplas threads em segundo plano) e registra uma função de *callback* a ser executada quando a operação terminar. Enquanto isso, o *event loop* fica livre para processar outras requisições. Este comportamento não bloqueante é particularmente vantajoso para uma API de geração de documentos, uma atividade inerentemente ligada a operações de I/O (leitura de templates, consulta de dados, escrita de arquivos). Ele permite que o sistema lide com um grande número de requisições

concorrentes com um consumo de recursos significativamente menor, o que é fundamental para construir APIs ágeis, responsivas e altamente escaláveis.

3.5.2. Persistência de dados flexível com MongoDB: O poder do NoSQL

O armazenamento dos templates de documentos apresenta um desafio particular que os bancos de dados relacionais tradicionais (SQL) não endereçam de forma ideal. Cada template criado por um usuário é, por natureza, uma entidade semiestruturada e dinâmica, possuindo um conjunto único e imprevisível de campos variáveis. Tentar modelar isso em uma estrutura relacional rígida, com tabelas e colunas pré-definidas, resultaria em esquemas excessivamente complexos ou ineficientes.

A escolha do MongoDB, um banco de dados NoSQL orientado a documentos, resolve este problema de forma elegante. O MongoDB armazena dados em formato BSON (uma representação binária de JSON), o que permite que cada documento dentro de uma mesma coleção possua uma estrutura completamente diferente. Essa flexibilidade de esquema (*schema-on-read*) é perfeitamente adequada para o caso de uso do projeto, pois permite armazenar a estrutura de cada template de forma nativa, sem a necessidade de pré-definir colunas ou criar tabelas de junção complexas. Essa abordagem não apenas simplifica o desenvolvimento e a manutenção, mas também permite que a plataforma evolua e suporte novos tipos de templates sem a necessidade de migrações de esquema dispendiosas e complexas.

Tabela 2 – Análise Comparativa: Modelo Relacional (SQL) vs. Modelo de Documentos (MongoDB) para Armazenamento de Templates.

Característica	Modelo Relacional (Ex: PostgreSQL)	Modelo de Documentos (MongoDB)
Estrutura de Dados (Schema)	Esquema rígido e pré-definido (<i>schema-on-write</i>). As tabelas e colunas devem ser definidas antes da inserção dos dados.	Esquema flexível e dinâmico (<i>schema-on-read</i>). Documentos na mesma coleção podem ter estruturas diferentes.
Flexibilidade para Novos Campos	Requer alteração na estrutura da tabela (ex: ALTER TABLE), o que pode ser	Nenhuma alteração de esquema é necessária. Novos campos podem

	uma operação custosa e complexa em produção.	ser adicionados a novos documentos a qualquer momento.
Escalabilidade Horizontal	Tradicionalmente escala verticalmente (aumentando os recursos de um único servidor). A escalabilidade horizontal (<i>sharding</i>) é possível, mas geralmente mais complexa de implementar.	Projetado para escalabilidade horizontal nativa através de <i>sharding</i> , distribuindo os dados por múltiplos servidores.
Complexidade de Consultas	Consultas para dados aninhados ou variáveis (como os campos de um <i>template</i>) exigiriam múltiplas junções (<i>JOINS</i>) ou modelos complexos (ex: <i>EAV</i>).	Dados relacionados e aninhados são armazenados juntos no mesmo documento, permitindo recuperação com uma única consulta, de forma mais simples e performática.
Adequação ao Caso de Uso	Menos adequado para dados semiestruturados e dinâmicos, como os <i>templates</i> , devido à sua rigidez estrutural.	Altamente adequado. O modelo de documento mapeia naturalmente a estrutura variável e aninhada dos <i>templates</i> de documentos.

Fonte: Elaborado pelo autor (2026).

3.5.3. Segurança e autenticação stateless via JWT

Para garantir a segurança e o controle de acesso à plataforma, foi adotado um modelo de autenticação *stateless* (sem estado) baseado em *JSON Web Tokens* (JWT). Em uma arquitetura *stateless*, o servidor não armazena informações sobre a sessão de um usuário após a autenticação. Em vez disso, cada requisição feita pelo cliente deve conter toda a informação necessária para que o servidor possa verificar sua identidade e autorização.

O JWT é um padrão aberto que permite a criação de tokens de acesso compactos e autossuficientes. Um JWT é composto por três partes, separadas por pontos:

1. Header: Contém metadados sobre o token, como o tipo (JWT) e o algoritmo de assinatura utilizado (ex: HS256).
2. Payload: Contém as "reivindicações" (*claims*), que são os dados sobre o usuário (como seu ID) e as permissões de acesso, além de metadados como a data de expiração do token.

3. Signature: Uma assinatura digital gerada a partir do *header*, do *payload* e de uma chave secreta conhecida apenas pelo servidor. Esta assinatura garante a integridade e a autenticidade do token.

O fluxo de autenticação funciona da seguinte maneira: o usuário se autentica com suas credenciais (login e senha); o servidor valida as credenciais e, se corretas, gera um JWT assinado e o retorna ao cliente. O cliente, então, armazena este token e o inclui em cada requisição subsequente a recursos protegidos, tipicamente no cabeçalho Authorization. O servidor, ao receber uma requisição, pode validar a assinatura do JWT usando sua chave secreta, sem a necessidade de consultar um banco de dados ou um armazenamento de sessão. Este modelo melhora a performance, reduz a carga no servidor e simplifica a escalabilidade, sendo ideal para arquiteturas distribuídas e de microsserviços.

3.6. CONFORMIDADE, INTEGRAÇÕES E HORIZONTES FUTUROS

Uma solução de software moderna não existe em um vácuo. Ela deve operar dentro de um quadro legal, interagir com um ecossistema de outras ferramentas e possuir uma visão clara de sua evolução futura. Esta seção final do referencial teórico posiciona a plataforma de automação de documentos dentro desses três contextos cruciais.

3.6.1. Implicações da Lei Geral de Proteção de Dados (LGPD)

A Lei Geral de Proteção de Dados Pessoais (LGPD), Lei nº 13.709/2018, estabeleceu um novo paradigma para o tratamento de dados pessoais no Brasil, com o objetivo de proteger os direitos fundamentais de liberdade e privacidade dos cidadãos. A lei se aplica a qualquer operação de tratamento de dados e exige que as organizações sigam dez princípios fundamentais, entre os quais se destacam a **finalidade** (o tratamento deve ter um propósito específico e informado), a **necessidade** (devem ser coletados apenas os dados mínimos necessários) e a **segurança** (devem ser adotadas medidas para proteger os dados contra acessos não autorizados e incidentes). O consentimento explícito do titular dos dados é, na maioria dos casos, a base legal que legitima o tratamento.

Neste contexto, uma plataforma de automação de documentos bem projetada pode se tornar uma poderosa ferramenta de conformidade com a LGPD. Ao centralizar e padronizar a geração de documentos que frequentemente contêm dados pessoais (como contratos de trabalho ou fichas cadastrais), o sistema permite a implementação de controles robustos. É possível garantir, por exemplo, que todos os contratos incluam cláusulas de consentimento padronizadas e atualizadas. Além disso, a plataforma pode oferecer recursos de controle de acesso granular, garantindo que apenas usuários autorizados possam criar ou visualizar documentos com informações sensíveis, e manter trilhas de auditoria (*audit logs*) detalhadas, registrando quem gerou, acessou ou modificou cada documento. Desta forma, a automação deixa de ser apenas uma ferramenta de eficiência e passa a ser um mecanismo ativo para mitigar riscos e auxiliar as organizações no cumprimento de suas obrigações legais.

3.6.2. Extensibilidade e o ecossistema de APIs

As aplicações contemporâneas são cada vez mais modulares e interconectadas, operando dentro de uma "economia de APIs" onde funcionalidades especializadas são consumidas de serviços externos através de Interfaces de Programação de Aplicações (APIs). A arquitetura da plataforma proposta foi concebida para ser extensível, permitindo integrações que enriquecem seu fluxo de trabalho e agregam valor ao usuário.

Dois exemplos práticos ilustram este potencial. O primeiro é a integração com serviços de assinatura digital, como DocuSign ou ZapSign. Ao invés de desenvolver internamente um sistema complexo para garantir a validade jurídica das assinaturas, a plataforma pode, via API, enviar um documento recém-gerado diretamente para a plataforma de assinatura, notificar os signatários e, após a conclusão, receber e arquivar a versão final assinada. Isso automatiza o ciclo de vida do documento de ponta a ponta.

O segundo exemplo é a integração com serviços de armazenamento em nuvem, como o Google Drive. Através da API do Google Drive, a aplicação pode oferecer ao usuário a opção de salvar os documentos gerados diretamente em sua conta pessoal ou corporativa na nuvem. Este processo é mediado pelo protocolo de autorização OAuth 2.0, que garante que a aplicação só tenha acesso ao Drive do

usuário após seu consentimento explícito, assegurando a privacidade e a segurança dos dados.

3.6.3. A próxima fronteira – Processamento inteligente de documentos (IDP)

A evolução natural de um sistema de automação é a incorporação de inteligência. A proposta de trabalho já vislumbra essa próxima fronteira, alinhada com o campo de Processamento Inteligente de Documentos (do inglês, *Intelligent Document Processing* - IDP). O IDP vai além da automação baseada em regras, utilizando Inteligência Artificial (IA), aprendizado de máquina (*machine learning*) e tecnologias como o Reconhecimento Óptico de Caracteres (OCR) para compreender e extrair dados estruturados a partir de fontes não estruturadas, como arquivos PDF, imagens ou e-mails.

A aplicação prática desta tecnologia no contexto do projeto é transformadora. Uma funcionalidade futura, como a de "Templates Inteligentes (IA)", permitiria que um usuário fizesse o upload de um documento existente — um contrato já preenchido, por exemplo. Um modelo de IA treinado para este fim analisaria o documento, identificaria os padrões e as informações variáveis (nomes, datas, valores, endereços) e, de forma autônoma, sugeriria a criação de um novo template, já com os marcadores de posição ({{...}}) inseridos nos locais corretos. Isso representaria um salto qualitativo fundamental: de um sistema que *executa* automações pré-definidas pelo usuário para um sistema que *cria* e *sugere* automações de forma inteligente, reduzindo drasticamente o esforço inicial de configuração e agregando um valor exponencial à experiência do usuário.

4. MATERIAL E MÉTODOS

4.1. NATUREZA E CLASSIFICAÇÃO DA PESQUISA

Para o desenvolvimento da plataforma de geração dinâmica de documentos, a metodologia adotada segue as diretrizes da pesquisa científica aplicadas à ciência da computação e aos sistemas de informação. A pesquisa é classificada sob quatro aspectos principais a seguir.

4.1.1. Quanto à natureza

A presente pesquisa classifica-se como aplicada. Esse tipo de pesquisa tem como característica o objetivo de gerar conhecimentos para aplicação prática, direcionados à solução de problemas específicos. Neste contexto, o trabalho sai do campo puramente teórico e propõe uma solução tecnológica real e funcional — um protótipo de plataforma web — visando resolver as ineficiências operacionais e os riscos associados à gestão documental manual.

4.1.2. Quanto à abordagem do problema

Quanto à forma de abordagem, a pesquisa caracteriza-se como qualitativa. O foco principal não é a quantificação estatística de um grande volume de dados de usuários, mas sim a compreensão profunda do problema e a validação da arquitetura tecnológica proposta. A análise qualitativa se dará pela avaliação de como as tecnologias escolhidas (Node.js, MongoDB e Handlebars.js) se comportam na resolução do problema de automação e na simplificação do uso para o usuário final.

4.1.3. Quanto aos objetivos

Do ponto de vista de seus objetivos, a pesquisa enquadra-se como exploratória e descritiva. É exploratória pois investiga o paradigma de motores de *template* (*template engines*) e os conceitos de Automação de Processos de Negócio (BPA) dentro do cenário da Transformação Digital. É descritiva porque documenta e detalha

minuciosamente a arquitetura de software implementada, as escolhas tecnológicas, os fluxos de trabalho desenvolvidos e as lógicas de integração.

4.1.4. Quanto aos procedimentos técnicos

Em relação aos procedimentos técnicos, o trabalho utiliza o método de desenvolvimento experimental (prototipagem). A pesquisa baseia-se na construção de um artefato tecnológico de software — um Produto Mínimo Viável (MVP) operando como *Software as a Service* (SaaS). Esse protótipo serve como instrumento principal para testar e validar na prática a viabilidade técnica da inserção de campos dinâmicos e da operação do motor de cálculo nos documentos.

4.2. ARQUITETURA DO SISTEMA E FERRAMENTAS UTILIZADAS

O desenvolvimento do protótipo funcional da plataforma web demandou a seleção de um conjunto de tecnologias modernas, focadas em escalabilidade, flexibilidade e facilidade de uso. A arquitetura da solução foi dividida em camadas lógicas, cada uma sustentada por ferramentas específicas.

4.2.1. Backend e Lógica de Negócios (Node.js)

A camada de servidor (backend) foi integralmente desenvolvida utilizando o Node.js. A escolha desse ambiente de execução JavaScript justificou-se pela sua arquitetura orientada a eventos e pelo modelo de Entrada/Saída (I/O) não bloqueante. Como a geração de documentos exige constantes operações de leitura de templates e escrita de arquivos finais, o modelo do Node.js permite o processamento de múltiplas requisições simultâneas de forma otimizada. O ecossistema Node.js também forneceu a base para o desenvolvimento da API RESTful que gerencia toda a comunicação entre a interface do usuário e o banco de dados.

4.2.2. Armazenamento e Persistência de Dados (MongoDB)

Para a camada de persistência de dados, optou-se pelo banco de dados NoSQL MongoDB. Em contraste com os bancos de dados relacionais tradicionais, o

MongoDB armazena dados em formato BSON (uma representação binária do JSON), o que confere ao sistema o recurso de esquema dinâmico (*schema-on-read*). Essa flexibilidade é um requisito fundamental para o projeto, uma vez que os templates criados pelos usuários possuem campos de variáveis dinâmicos e imprevisíveis, tornando a modelagem relacional excessivamente complexa e rígida.

4.2.3. Motor de Templates (Handlebars.js)

O núcleo da geração dinâmica de documentos foi construído sobre a biblioteca Handlebars.js. Este motor de templates foi selecionado devido à sua filosofia de design *logic-less* (sem lógica complexa embutida), que mantém a sintaxe limpa e declarativa. O Handlebars compila os modelos de documentos criados pelos usuários combinando-os com objetos de dados JSON (fornecidos via API) para renderizar a versão final do documento. Essa abordagem viabilizou a criação de uma interface intuitiva onde usuários sem conhecimento em programação conseguem inserir delimitadores simples (chaves duplas) para representar variáveis dinâmicas em seus textos.

4.2.4. Segurança e Autenticação (JWT)

Para garantir a proteção dos dados dos usuários e a conformidade com as melhores práticas de segurança (essenciais em sistemas que lidam com informações sensíveis e LGPD), a autenticação da plataforma foi implementada utilizando JSON Web Tokens (JWT). A adoção de um modelo *stateless* dispensou o armazenamento de sessões no servidor, permitindo que cada requisição à API fosse validada através de uma assinatura criptográfica contida no token. Isso não apenas reforçou a segurança do acesso, mas também otimizou a performance da aplicação.

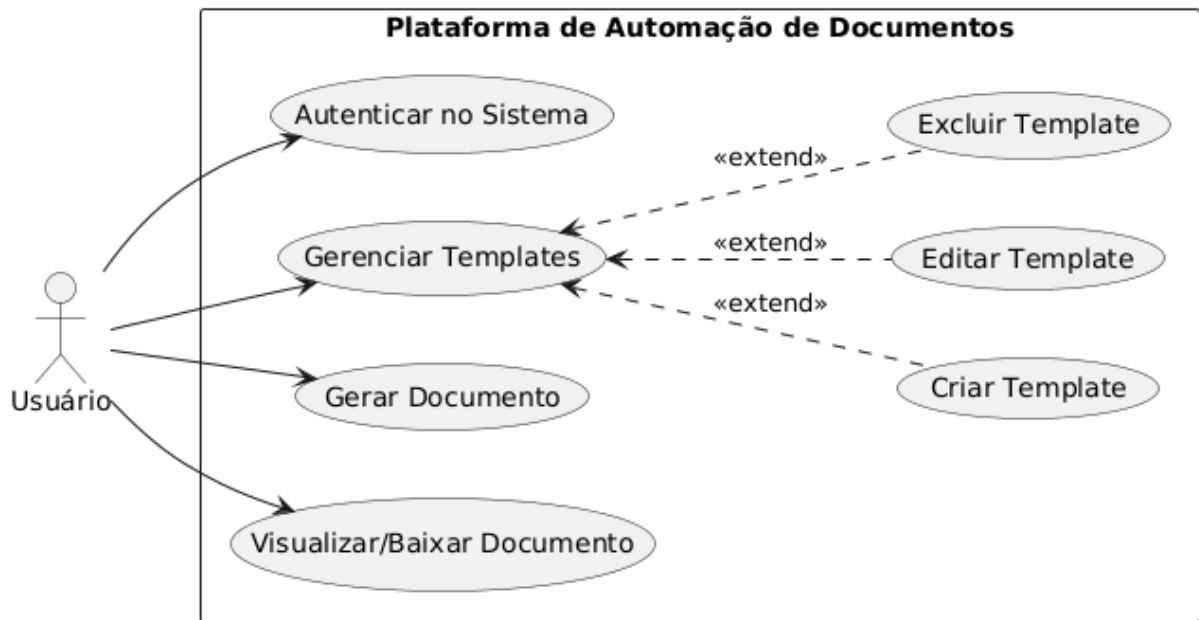
4.3. MODELAGEM DO SISTEMA

Para a representação visual dos requisitos funcionais e da estrutura lógica de dados da plataforma de automação, adotou-se a Linguagem de Modelagem Unificada (UML). A modelagem foi dividida em duas perspectivas principais: a interação do usuário com o sistema e a estruturação interna das entidades de domínio.

4.3.1. Diagrama de Casos de Uso

O Diagrama de Casos de Uso tem como objetivo mapear as principais funcionalidades que o sistema oferece sob a ótica do usuário final. Sendo uma plataforma projetada para que pessoas sem conhecimento técnico avançado possam gerenciar seus modelos, o fluxo de interação foi simplificado.

Figura 1 – Diagrama de Caso de Uso

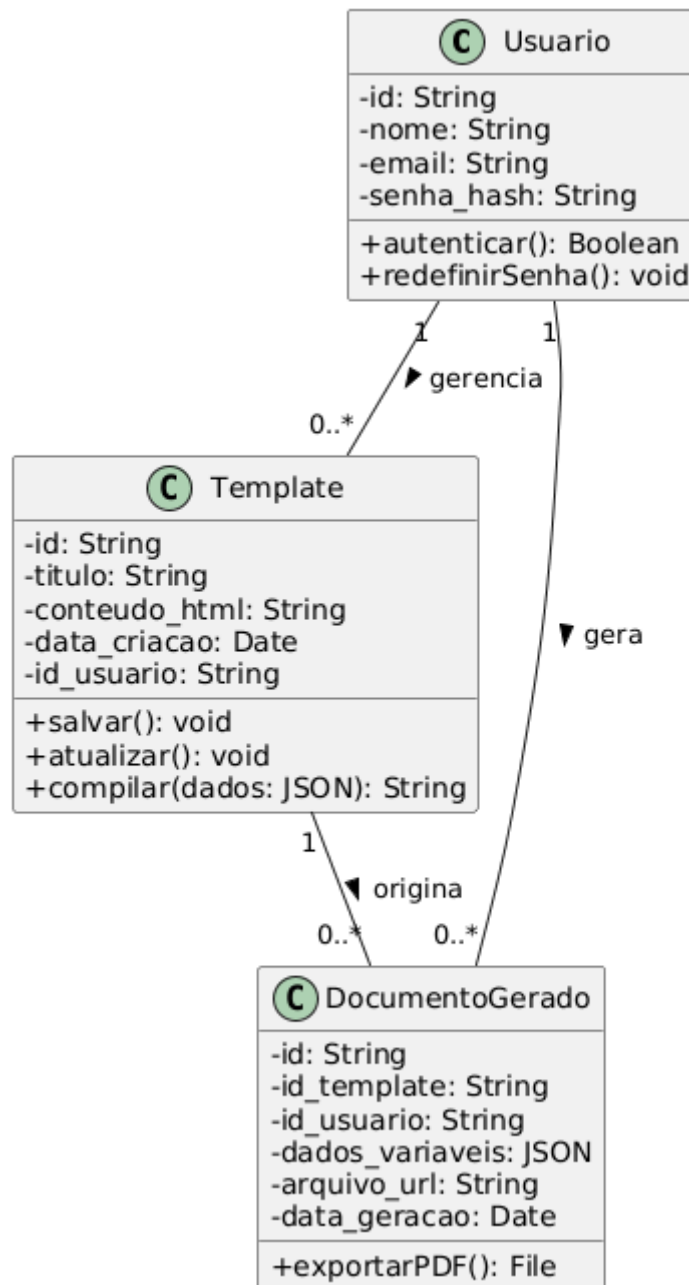


Fonte: Elaborado pelo Autor (2026).

4.3.2. Diagrama de Classes

O Diagrama de Classes ilustra a estrutura de dados e as entidades de negócio que trafegam entre o backend em Node.js e o banco de dados MongoDB. Apesar da natureza flexível do NoSQL, as entidades principais possuem uma estrutura base para garantir a integridade da aplicação.

Figura 2 – Diagrama de Classes



Fonte: Elaborado pelo Autor (2026).

4.4. ETAPAS DE DESENVOLVIMENTO

Para a construção do protótipo funcional da plataforma web, o ciclo de vida do desenvolvimento de software foi estruturado em três etapas fundamentais, baseadas em metodologias ágeis e iterativas. Essa abordagem permitiu a entrega contínua de valor e a validação constante das lógicas de automação de documentos propostas.

4.4.1. Levantamento e Análise de Requisitos

A primeira etapa consistiu na modelagem conceitual do sistema e no levantamento de seus requisitos funcionais e não funcionais. Nesta fase, definiu-se que a aplicação operaria com o modelo de motor de templates, exigindo uma linguagem de marcação baseada em chaves duplas `{{ }}` para facilitar a usabilidade por usuários sem conhecimento técnico. Também foram estruturados os modelos de dados (estruturas JSON) que alimentariam o MongoDB e definidos os fluxos de segurança que exigiriam a implementação do JWT para o controle de acesso e conformidade com os princípios da LGPD.

4.4.2. Implementação e Codificação (Construção do MVP)

A fase de implementação focou na construção do Produto Mínimo Viável (MVP). O desenvolvimento foi iniciado pela configuração do servidor backend utilizando Node.js, tirando proveito de sua arquitetura não bloqueante para o gerenciamento eficiente de rotas e requisições concorrentes. Em seguida, realizou-se a integração com o banco de dados NoSQL MongoDB, criando as coleções necessárias para armazenar de forma flexível as estruturas dinâmicas dos templates (*schema-on-read*). Por fim, o motor de compilação do Handlebars.js foi integrado à API, configurando as lógicas de substituição de variáveis e o motor de cálculo responsável por processar as operações dinâmicas diretamente nos documentos finais.

4.4.3. Testes e Validação do Sistema

A etapa final do ciclo de desenvolvimento englobou a realização de testes para assegurar a confiabilidade e a precisão do sistema. Os testes concentraram-se na validação da API (*endpoints*), verificando a correta emissão e validação dos tokens JWT nas rotas protegidas. O núcleo dos testes focou na capacidade do motor de renderização: foram criados diversos templates de teste com variados níveis de complexidade (incluindo laços de repetição e estruturas condicionais suportadas pelo Handlebars) para garantir que a injeção de dados JSON ocorresse sem falhas, resultando em documentos finais consistentes, sem omissões de dados e devidamente padronizados.

5. RESULTADOS E DISCUSSÃO

5.1. VALIDAÇÃO DA ARQUITETURA E DO MODELO DE DADOS

A escolha de uma arquitetura baseada em Node.js e MongoDB mostrou-se, durante a fase de modelagem, a solução mais resiliente para o problema da geração de documentos em massa. Observou-se que o modelo orientado a eventos do Node.js é capaz de sustentar o fluxo de Entrada/Saída (I/O) necessário para a leitura de múltiplos templates simultâneos sem o bloqueio da *thread* principal, o que é crítico para uma aplicação de alta performance.

Quanto à persistência, a utilização do MongoDB validou o paradigma de esquema flexível (*schema-on-read*). Diferente de modelos relacionais que exigiriam tabelas complexas para campos variáveis, o formato BSON permitiu que cada *template* de documento armazene sua própria estrutura de marcadores de forma nativa e eficiente. Essa flexibilidade é o que sustenta a proposta de valor de permitir que usuários leigos criem modelos sem a intervenção da equipe de TI.

5.2. PLANO DE TESTES E GARANTIA DE QUALIDADE

Para assegurar a confiabilidade do sistema, estabeleceu-se um protocolo de testes dividido em três frentes principais:

5.2.1. Testes de Injeção de Dados e Renderização

Utilizando a biblioteca Handlebars.js, o foco é garantir que a substituição de chaves duplas (ex: `{{nome}}`) ocorra de forma íntegra, mesmo em templates com estruturas condicionais e laços de repetição complexos.

5.2.2. Validação de Segurança (JWT)

O sistema foi submetido a testes de interceptação de rotas para garantir que nenhum recurso protegido seja acessado sem um token válido e assinado. A

abordagem *stateless* mostrou-se eficiente para reduzir a carga de processamento no servidor de autenticação.

5.2.3. Cenários de Escalabilidade

Planeja-se a execução de testes de estresse para mensurar o tempo de resposta da API na geração simultânea de lotes de 100, 500 e 1.000 documentos, validando a capacidade do Node.js em gerenciar requisições concorrentes.

5.3. GANHOS OPERACIONAIS E CONFORMIDADE ESTRATÉGICA

A discussão dos resultados projeta uma redução drástica no chamado "gap silencioso" de performance nas organizações. Ao converter processos manuais lentos em fluxos automatizados, espera-se mitigar os 80% a 90% de riscos associados a erros humanos na digitação de dados sensíveis.

Além da eficiência, a plataforma atua como um mecanismo de compliance. Através de trilhas de auditoria e do controle granular de acesso, a solução facilita a conformidade com a Lei Geral de Proteção de Dados (LGPD), assegurando que dados pessoais em contratos e laudos sejam tratados apenas por usuários devidamente autorizados. A padronização resultante fortalece não apenas a segurança jurídica, mas também a identidade visual das marcas que utilizarem o sistema.

6. CONSIDERAÇÕES FINAIS

Parte do texto que apresenta os resultados correspondentes aos objetivos ou hipóteses levantadas na introdução. O mesmo se aplica se o resultado é a proposta de um produto ou processo.

Descreve de forma resumida o que se aprendeu sobre o tema, até mesmo propostas para outros trabalhos referentes ao assunto. Deve estar coerente com o desenvolvimento e relacionado à introdução. Pode ainda estabelecer relações com outros fatos referentes à mesma matéria.

Em trabalhos acadêmicos, o termo “Conclusão” é substituído por “Considerações Finais”.

7. SUGESTÃO TRABALHOS FUTUROS

REFERÊNCIAS

ALVES, R. de B. **Ciência criminal**. Rio de Janeiro: Forense, 1995.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6023**: informação e documentação: Referências: Elaboração. Rio de Janeiro, 2002.

CERVO, A. L.; BERVIAN, P. A.; SILVA, R. da. **Metodologia científica**. 6. ed. São Paulo: Pearson Prentice Hall, 2007.

MINORELLO, D.; REHDER, W. da S. **Photoshop CS3**: edição profissional de imagens e gráficos. São Cruz do Rio Pardo: Viena, 2008.

UNIVERSIDADE DE SÃO PAULO. **Catálogo de teses da Universidade de São Paulo, 1992**. São Paulo, 1993. 467 p.